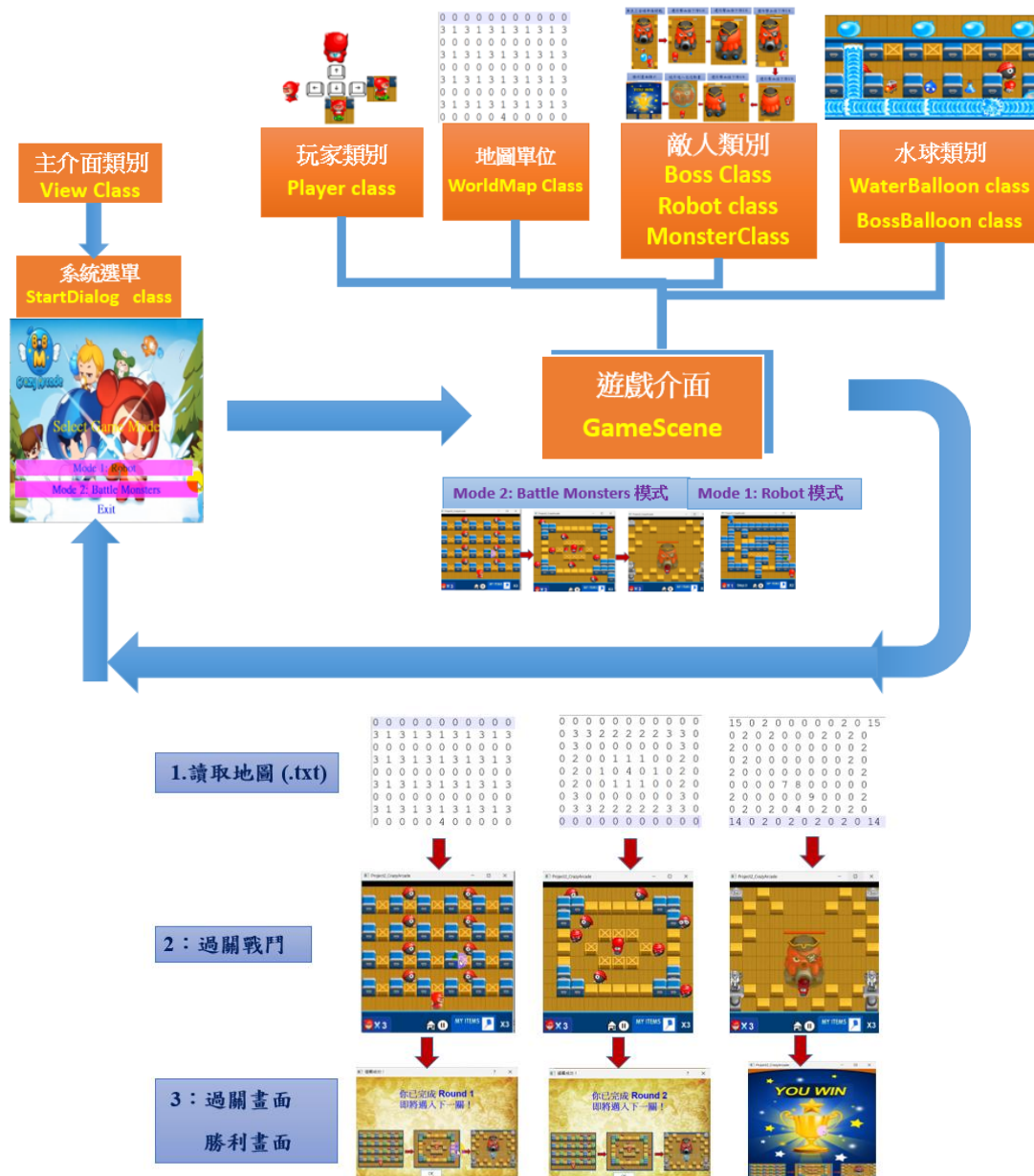


計算機概論（二）

Project2

Crazy Arcade (爆爆王) Report



學號姓名：E24134778 彭靖凱、E24136128 丁宗泓

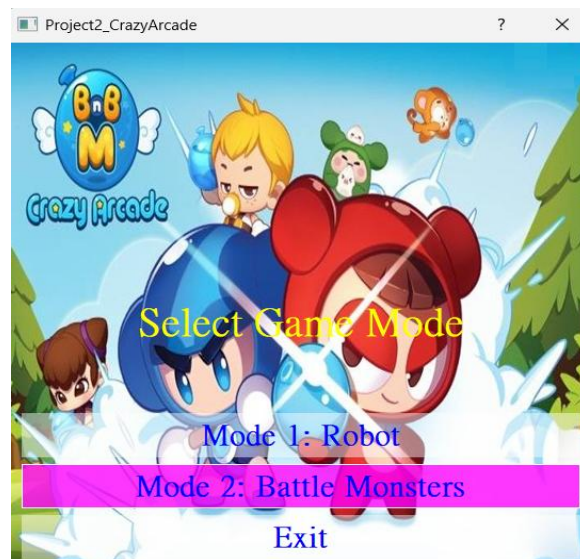
授課教授：陳盈如 教授

實作說明（Implementation Description）

本專案《Crazy Arcade（爆爆王）》依據 Project2 規範，完整實作兩種遊戲模式：Robot 模式與 Battle 模式，並使用 Qt 的 QGraphicsScene 與 QTimer 作為主體架構，輔以 BFS 演算法進行 AI 尋路、物件互動判斷與水球爆炸系統。

整體遊戲流程由 main.cpp 負責開啟應用程式，並播放動畫式啟動畫面（Splash Screen），其後進入 StartDialog 主選單，StartDialog 提供使用者選擇三項選項：

- Mode 1: Robot 模式
- Mode 2: Battle Monsters 模式
- Exit 離開



StartDialog 主選單 Robot 模式與 Battle 模式

Robot 模式（模式一）

Robot 模式中，玩家角色維持靜止，由 AI 控制的機器人主動移動並追蹤玩家。起點為機器人的地圖座標，終點為玩家初始位置，主要透過 bfsFindPath(QPoint start, QPoint end) 實作路徑搜尋。搜尋過程中，會避開 type=3 的磚塊（不可破壞牆）與水球爆炸區域（Danger Zone）。

若路徑中有磚塊 `type=1`（可推動），AI 會判斷是否可以推開後，再次進行 BFS 以確認是否存在可通行路線。若無法推動或路徑仍不通，則觸發「放水球」機制，透過 AI 在合適位置放置水球，並利用其爆炸破壞 `type=1` 的障礙物。這段爆炸機制使用 `QTime` 計時，水球 3 秒後爆炸，並對上下左右一格造成影響。

若最短路徑中出現磚塊 `type=2`（固定可破壞），則 AI 直接放置水球來炸毀該磚塊。而遇到 `type=3`（不可破壞磚塊）則必須完全繞道避開。AI 會同步評估爆炸風險，若判斷放置水球後無法於 3 秒內逃出爆炸範圍，則選擇暫停行動，直到下一次更新週期再次判斷。

整個 Robot 模式會透過定時器更新畫面與邏輯，每 10 毫秒刷新一次位置與水球狀態，同時在介面下方顯示當前機器人行動步數，供使用者參考。

Robot 控制相關

1. `WorldMap::loadMap()`

讀取 `.txt` 地圖檔案（例如 `:/maps/testmap.txt`）。

- (1) 解析每一行文字內容，其中每一個字元或整數代表一種地圖元素：
- (2) 例如 0 表示空格，1 代表牆，2 可能代表箱子等。
- (3) 儲存在 `mapData` 二維陣列中，建立對應的 `MapObject` 物件。
- (4) 同時記錄特殊位置：
 `RobotSt`：機器人起始位置。
 `playSt`：玩家起始位置

2. `robot (Robot*)`

→ 機器人實體，在 `GameScene::Robot(&m_STgame.world, m_STgame.world.RobotSt)` 中初始化。

3. `QTimer * m_Robot_timer`

→ 控制 Robot 行為的計時器，每 10 毫秒呼叫一次。

4. `void GameScene::updateRobot()`

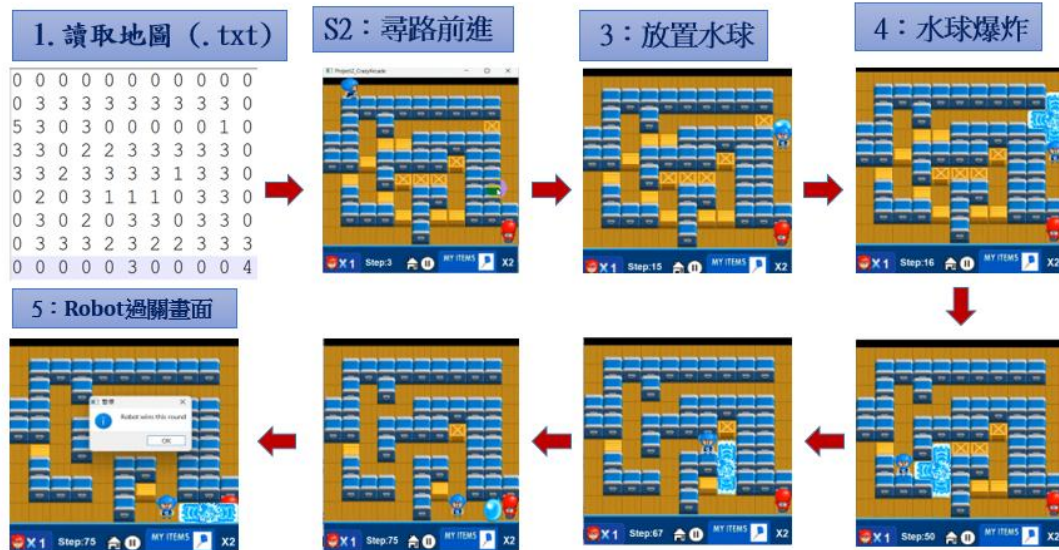
函式用於在機器人模式下更新機器人的行為。其邏輯包括：判斷是否該放置水球；如果目前路徑 `path` 已完成，則重新計算路徑至玩家位置；若尚有路徑則逐步移動機器人並更新位置與步驟數，並定期切換動畫圖片或方向指示。此外，當條件達成時也會觸發水球放置與爆炸效果。整體邏輯模擬智慧機器人追蹤玩家並適時攻擊的行為模式。

5. robot-> bfsFindPath (start, end)
→ 執行 BFS 演算法，搜尋到玩家的最短安全路徑（避開牆壁與水柱區）。
6. robot-> getFreeCell ()
→ 搜索起點附近的安全格子，常用於放置水球後的逃跑路徑。玩家變數與行為
7. void GameScene::checkPlayerCollideRobot()
→ 若玩家的 m_player_index 為 1（可能表示無敵或特殊狀態），則直接 return。建立 RobotRect 表示機器人邊界。呼叫
playerRect.intersects(RobotRect)若玩家與機器人碰撞，顯示訊息框
「Robot wins this round」。呼叫 clear() 清空場景，並觸發
showGameOverAndExit(1) 結束遊戲。
8. player->life (int)
→ 玩家生命值，在 Mode 1 模式中設為 1，任一爆炸命中即 Game Over。

水球爆炸控制

1. BossBalloon 類別（由 robot 產生）
→ 透過 spawnTime 記錄放置時間。若超過 3 秒 觸發爆炸。。
 2. GameScene::draw_Waterballoon()
→ 每幀掃描所有水球，檢查是否到達爆炸時間，並觸發爆炸動畫。
 3. GameScene::draw_WaterColumn()
→ 計算爆炸波及範圍，若範圍內包含 player 或 robot 則設定其死亡標誌。
4. 結束條件 (End Conditions)
1. 玩家有 1 命 (life = 1)條機器人有 1 命 。
 2. 玩家被機器人攻擊到（例如水柱、碰撞等）

Mode 1: Robot 模式



Battle 模式 (模式二)

Battle 模式中，玩家可自由移動，並使用方向鍵操作角色，按下空白鍵放置水球。放置後 3 秒，水球會產生爆炸，向上下左右延伸一定格數（可透過道具增加），形成範圍傷害。若水球爆炸波及其他尚未爆炸的水球，會立即引爆產生「連鎖爆炸」效果。這段邏輯會偵測方向上是否有障礙物（如磚塊、牆），若有則中斷爆炸路徑。

玩家初始擁有三條命與三支針。若被水球困住超過 3 秒未逃脫，則會失去一命，並重生於初始點。同時，若擁有針可按 Shift 鍵破球逃脫。玩家也可收集場上的道具，如鞋子（增加移動速度）、水球強化藥水（增加爆炸範圍）、針等，皆由系統隨機掉落。

每一關地圖不同，並由 testmap1.txt、testmap2.txt 與 testmap3.txt 載入。前兩關為基本怪物關卡，怪物有固定路徑與隨機行走策略；第三關為 Boss 關卡，出現大型章魚怪物，具有發射「Boss 水球」的能力。Boss 水球會沿直線飛行、撞牆反彈，並有 3 秒爆炸延遲，同時擁有額外判斷邏輯（如回彈方向、與玩家碰撞等）。

整體來說，Battle 模式透過 drawBG() 與 drawObj() 呈現場景與物件，並搭配 drawUI() 實時顯示生命與道具數量。每一輪完成後會跳出過關視窗，並自動切入下一關地圖與怪物組態，實現三輪遊戲流程。

補充設計與延伸功能

遊戲支援 txt 地圖格式自訂功能，允許變更玩家、機器人、怪物與磚塊位置，並透過物件 type 控制不同特性。每一種物件皆有對應圖片與圖層順序，透過 Z 值控制繪圖順序。此外，本作品亦整合動畫啟動畫面與背景音效，使整體更具互動性與遊戲感。

Battle 模式 控制相關

第一步：m_STgame.world.loadMap("/:maps/testmap1.txt")讀取地圖 (.txt)

- 使用 .txt 檔匯入每回合的地圖結構每一關地圖不同，並由 testmap1.txt、testmap2.txt 與 testmap3.txt 載入。
- 每張地圖代表不同回合：
 - (1) initMonstersRound1() Round 1：橫向移動怪物
 - (2) initMonstersRound2() Round 2：順／逆時針繞圈移動怪物
 - (3) initBoss(); Round 3 (Boss)：Boss 出現、BOSS 專屬地圖與挑戰

第二步：GameScene::updateBattle()（遊玩過程）過關戰鬥

- (1) 玩家使用 **方向鍵移動**、**空白鍵放水球**、**Shift 用針逃生**
 - (2) 地圖會出現怪物或 Boss，根據關卡設定移動方式不同
 - (3) 玩家需擊敗所有怪物/Boss 才能進入下一關
- 玩家與畫面更新

控制功能說明

PlayerMovement()：處理玩家移動輸入。

drawBG()：繪製背景。

drawUI()：更新生命與道具數量顯示。

drawObj()：繪製地圖物件。

draw_player()：繪製玩家角色。

Round 1 & 2 處理

draw_monsters()：繪製怪物。

checkMonsterHitByFlame()：怪物是否被火焰打中。

checkPlayerCollideMonster()：玩家與怪物碰撞檢查。

checkPlayerInExplosion()：玩家是否站在爆炸範圍內。

Round 3 Boss 關卡處理

若 TouchBossFlag 為真 → showVictory()。

bossEntryAnimation()：執行 Boss 進場動畫。

updateBossAI()：控制 Boss 移動與行為。

draw_boss()：繪製 Boss。

BossBalloon 控制

fireBossBalloon()：根據玩家位置射出水球。

updateBossBalloons()：水球路徑更新與碰撞檢查。若即將命中 → 強制提前爆炸。

Draw_BossWaterColumn()計算爆炸波及範圍，若範圍內包含 player 或 boss 則設定其死亡標誌。。

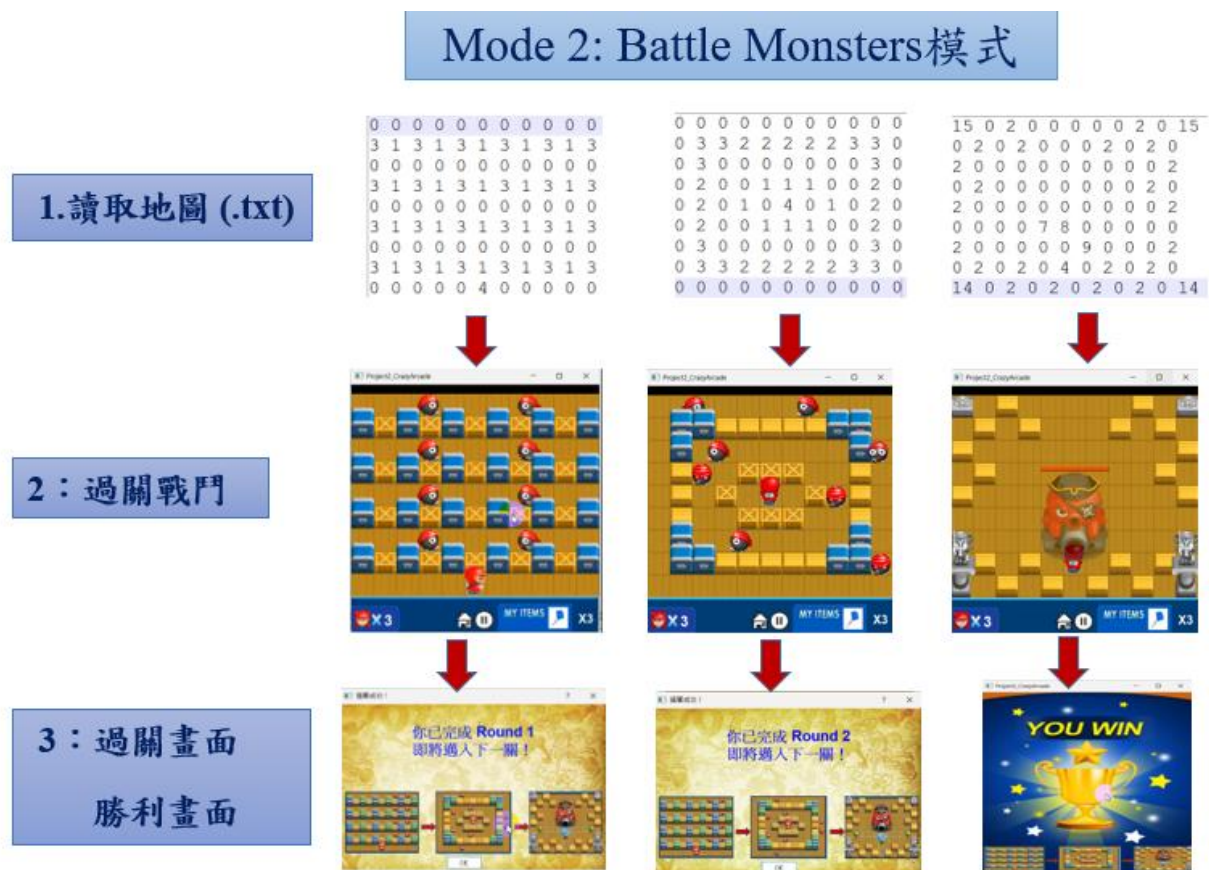
第三步：顯示畫面

1. 完成每回合後顯示：

(1) RoundCompleteDialog:show「你已完成 Round X，即將進入下一關」

2. 第三回合 (Boss) 勝利後顯示：

(1) void GameScene::showVictory()：YOU WIN 畫面，標誌最終過關成功

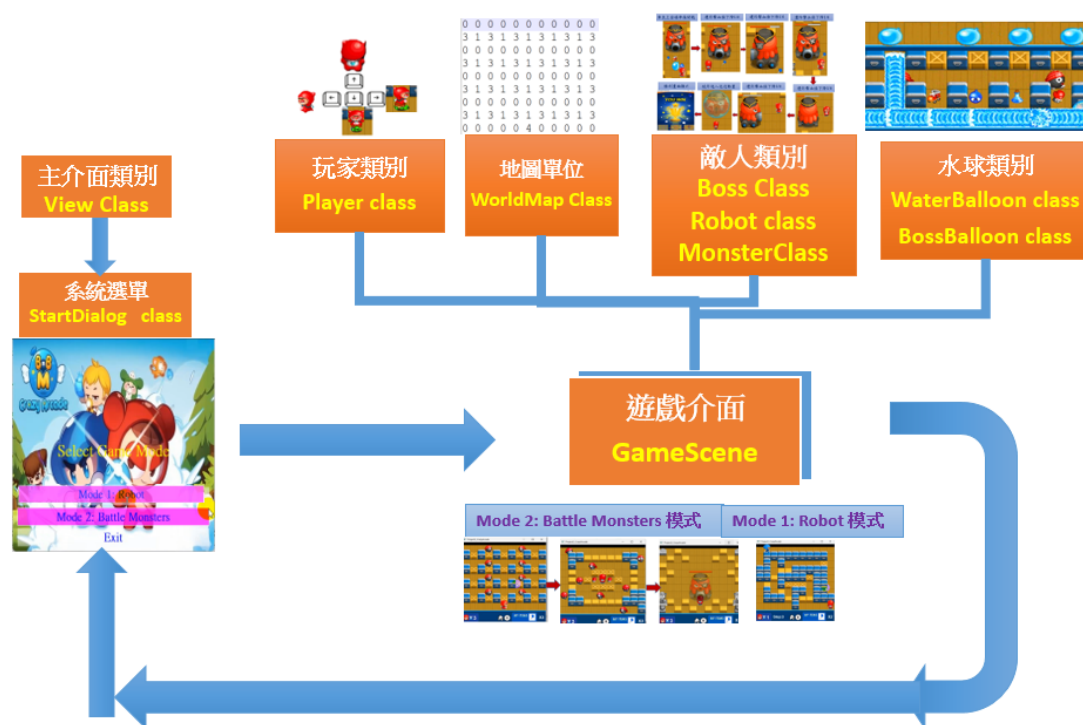


系統架構設計

本遊戲專案基於 C++ 與 Qt 框架，採用 QGraphicsView/QGraphicsScene 架構模型 搭配事件驅動與計時器邏輯，實作出一款擁有動畫、AI、自訂地圖(txt WorldMap)、道具系統與多種角色互動（如 Robot、Player）的 2D 策略遊戲。

《Crazy Arcade（爆爆王）》支援 Robot 模式與 Battle 模式，兩者皆共用核心架構，並透過參數切換與地圖變換支援完整流程控制。

- Framework:



Crazy Arcade（爆爆王） 模組總表

模組名稱	功能說明
main.cpp	是程式的進入點，負責啟動應用程式與初始畫面。首先建立 QApplication 物件，接著顯示帶有動畫效果的啟動畫面（Splash Screen），背景圖為 start_menu.jpg。動畫採用 QPropertyAnimation，從小放大顯示圖片。之後以 QTimer 延遲顯示 StartDialog 主選單，讓玩家選擇遊戲模式。根據選擇的結果，載入對應的 View（遊戲畫面），啟動遊戲主流程。整體流程清楚銜接 UI、動畫與主選單，是遊戲啟動與模式切換的核心橋樑。
StartDialog	類別是遊戲的主選單介面，繼承自 QDialog，提供玩家選擇遊戲模式或退出的選項。其介面包含標題與三個按鈕：「模式一（Robot）」、「模式二（Battle Monsters）」及「退出」。
View	類別繼承自 QGraphicsView，是遊戲的主要畫面顯示介面，負責呈現 GameScene。建構時會根據傳入的模式（mode）建立對應的 GameScene，並設定場景尺寸（550x550）、黑色背景、不顯示捲軸。它是遊戲畫面的容器，會隨模式變換載入不同內容，如機器人模式或怪物戰鬥模式。View 也與遊戲的邏輯流程緊密連動，是場景切換與畫面顯示的關鍵類別。整體設計簡潔，專注於畫面承載與設定，讓遊戲內容透過 GameScene 動態呈現。
GameScene	GameScene 類別繼承自 QGraphicsScene，是遊戲的主要邏輯與畫面控制核心。根據模式（機器人或怪物戰）初始化對應地圖與角色，並定時更新遊戲狀態（如移動、爆炸、碰撞）。負責繪製背景、角色、水球、UI 等圖像，並管理 Boss、怪物、玩家等互動行為。透過 QTimer 控制動畫與事件循環，提供完整的場景管理與遊戲流程。亦包含關卡進度處理（如通關或失敗）、水球爆炸連鎖與特效呈現，是整個遊戲的核心場景控制器。
WorldMap	WorldMap 類別是遊戲地圖的核心管理器，負責地圖格子資料與遊戲物件的配置。地圖為 9x11 格，每格對應一個 MapObject，代表牆、角色、水球或道具等。支援從檔案載入地圖、重設狀態、物件放置與移動等功能。提供邊界檢查（isInBounds）、是否可行走（isWalkable）、是否為

	危險區 (isInDangerZone) 等判定方法。也管理爆炸狀態，避免同格子多次處理。此類別為遊戲場景與邏輯互動的基礎，與玩家、機器人、怪物等系統密切整合。
Robot	類別代表機器人角色，擁有自動尋路與放置水球能力。內含 bfsFindPath 方法，使用廣度優先搜尋 (BFS) 從起點到終點尋找安全路徑，會避開牆與爆炸區域。 getFreeCell 方法找出起點附近安全格子，利於機器人迴避與移動。成員變數包含機器人位置、路徑、爆炸逃逸路徑與行走步數等資訊。isPathOutsideFreeCells 用於判斷目前路徑是否落在安全區域外。整體設計結合 AI 尋路與遊戲地圖互動，是自動化行動與策略決策的核心元件。
Boss	代表最終關卡的首領，包含血量 (HP)、攻擊力 (Water_Pow)、水球數量等屬性，並具備進場動畫、受傷標記等狀態控制
Monster	一般敵人，紀錄其位置、方向與圖片索引。支援靜態怪 (第一關) 與路徑移動怪 (第二關)
Player	類別代表玩家角色，控制其移動、放置水球、與怪物與 Boss 互動。支援上下左右移動並包含障礙判定 (如牆、水球、推磚、道具觸發)。能放置水球並利用手套投擲穿牆。管理玩家狀態如生命值、速度、陷阱、月行模式與道具效果。能對章魚王 Boss 發射水球，追蹤與處理水球碰撞、反彈與路徑變化。此類別整合玩家操作、地圖互動與戰鬥行為，是遊戲進行的核心操作控制者。
WaterBalloon	水球物件，支援放置、倒數計時、範圍爆炸與連鎖爆炸判斷
BossBalloon	Boss 彈道水球，具有彈跳路徑與延遲爆炸邏輯
Sounds	管理背景音樂與音效，包括放水球、爆炸、勝敗提示等
drawUI()	即時更新畫面上的生命值、道具數量與當前關卡提示
drawBG() / drawObj()	繪製背景格子與所有場景物件 (磚塊、怪物、角色、道具等)
RoundCompleteDialog	當玩家完成一個關卡時會跳出來，提示玩家「過關成功，即將進入下一關」。
view.cpp/h	對應 View 模組邏輯，負責滾動條、尺寸與場景設置

《Crazy Arcade (爆爆王)》設計的模組層級結構樹狀圖

模組層級結構 (main.cpp → View → GameScene)

```
main.cpp
└─ StartDialog
   └─ View (QGraphicsView)
      └─ GameScene (QGraphicsScene)
         ├── STgame (整合資料與控制器)
         │   ├── Player
         │   │   ├── 道具屬性 (針、鞋子、水球數)
         │   │   └─ 狀態控制 (移動、放水球、被困)
         │   ├── Robot (AI 控制類, Robot 模式用)
         │   ├── WorldMap (讀取 txt 地圖)
         │   │   └─ MapObject[][] (地圖格子陣列)
         │   ├── WaterBalloon (水球邏輯)
         │   └─ BossBalloon (Boss 射擊水球)
         ├── drawUI() (介面更新函數)
         ├── drawBG() / drawObj() (背景與物件繪圖)
         ├── checkPlayerCollideMonster() (怪物碰撞判定)
         ├── checkMonsterHitByFlame() (怪物受傷判定)
         ├── checkPlayerInExplosion() (玩家被炸判定)
         ├── checkBossHitByExplosion() (Boss 受傷判定)
         ├── updateRobot() (主場景更新, Robot 模式)
         ├── updateBattle() (戰鬥更新, 怪物/Boss 模式)
         ├── showGameOverAndExit(int) (顯示遊戲結束)
         ├── returnToMainMenu() (返回主選單)
         ├── Sounds (音效控制器)
         │   ├── playBackgroundMusic()
         │   └─ playSoundEffect(QString name)
         ├── MyImage (圖片資源載入器)
         │   ├── loadPixmap()
         │   └─ QPixmap[] (角色、Boss、道具等圖片)
         └─ QTimer (驅動定時更新與爆炸判斷)
```

1. 架構採用說明

Qt 是跨平台的 GUI 程式設計框架，在本專案中，選擇使用 **View**(QGraphicsView) 與 **GameScene**(QGraphicsScene) 為主要畫面容器，優點如下：

- 支援大量圖形物件的繪製與更新
- 支援圖層景深控制，可繪製背景、角色、道具等多層內容
- 與定時器 QTimer 搭配，可實作動畫、碰撞與爆炸等時間邏輯

整體畫面由三層組成：

1. 背景層：地圖背景、地板磚圖
2. 物件層：玩家、怪物、機器人、水球與磚塊
3. UI 層：生命數、道具數、主選單與暫停按鈕等

2. 專案主流程結構 StartDialog

整體流程以 main.cpp 為啟動點，依序開啟動畫頁面、主選單與主視窗畫面。
以下為主結構說明：

- **main.cpp**：遊戲入口點，播放啟動畫面後進入 StartDialog。
- **StartDialog**：主選單畫面，使用者選擇 Model (Robot) 或 Mode2 (Battle) 後，初始化 View 與 GameScene。
- **View (QGraphicsView)**：畫面容器，固定大小為 550x550，避免滾動，與 GameScene 綁定。
- **GameScene (QGraphicsScene)**：遊戲主邏輯場景，依照 mode 參數切換 Robot/Battle 模式，負責載入地圖、初始化角色與控制整體畫面。

3. 地圖與物件資料管理：WorldMap class

地圖檔為文字檔 .txt 格式，內含 11x9 的格子陣列，代表不同物件的 type：

- (1) 0：空地
- (2) 1：可推可炸的磚塊
- (3) 2：可炸但不可推的磚塊

- (4) 3：不可破壞磚塊（牆）
- (5) 4：玩家起始點
- (6) 5：機器人起始點

地圖由 `WorldMap::loadMap()` 載入後，建立對應的 `MapObject` 物件並置入 2D 陣列 `map[y][x]` 中，每個格子皆有位置、`type`、圖像物件 `sprite` 與碰撞邊界資訊 `Boundary`。同時記錄玩家與機器人的起始座標以供初始化角色使用。

4. Robot 模式架構設計 Robot 類別

Robot 模式核心為 AI 控制的機器人，其關鍵在於 BFS 路徑搜尋與策略判斷。

(1) Robot 類別設計：`class Robot`

- 甲、`bfsFindPath(start, end)` 為核心尋路函數
- 乙、自動避開 `type=3`、爆炸區，若路徑中含有 `type=1` 則視情況嘗試推動或放水球
- 丙、若無路可走，回傳空路徑並暫停動作

(2) AI 放水球策略：`void Player::RobotplaceBall(QPoint ps)`

- 甲、判斷目標格子為磚塊 `type=1` 或 `type=2`，則在鄰近格子放置水球
- 乙、利用 `BossBalloon` 類別模擬機器人水球，記錄放置時間與爆炸邏輯
- 丙、放置後會啟動 `QTimer`，每 10 毫秒更新一次，持續追蹤是否爆炸、是否清除磚塊

(3) 機器人避險策略：`QVector<QPoint> Robot::getFreeCell(QPoint start)`

- 甲、若目前格子會在水球爆炸時被波及，AI 不會前進並暫停行動
- 乙、若已放置水球，優先尋找 `escape path` 離開爆炸範圍，否則強制停留

(4) Robot 遊戲主要更新：`void GameScene::updateRobot()`

Model 1 Robot 遊戲主要更新函式（每次 `m_Robot_timer` 觸發時執行機器人移動與攻擊判斷。

包含

```
// 檢查玩家是否被爆炸擊中，如果是就觸發死亡（GameOver）
checkPlayerInExplosion();
// 檢查玩家是否與機器人發生碰撞，若有也可能觸發死亡或失敗
checkPlayerCollideRobot();
// 檢查機器人是否被水球爆炸波及，若被擊中則視為死亡
checkRobotInExplosion();
```


5. Battle 模式架構設計

- (1) Battle 模式主要為玩家操作與怪物互動，三個關卡三個關卡（Round1 → Round2 → Round3）同樣循序進行，但以怪物為主。
- (2) 玩家操控主角移動、放水球打怪。
- (3) 生命值、道具等 UI 與場景元素由 drawUI()、drawObj()、drawBG() 等函數構成。
- (4) 每一回合中每種道具隨機出現（如針、鞋子、水球道具等）
- (5) 玩家需使用水球擊敗所有怪物才能過關

其架構重點如下：

1. 玩家控制（Player 類別）：

- (1) void GameScene::keyPressEvent(QKeyEvent *event):使用方向鍵控制玩家移動
- (2) m_STgame.m_StevenPeng.placeBall():按空白鍵放置玩家水球
- (3) m_STgame.m_StevenPeng.m_player_index=2:按 Shift 鍵被困水球時使用針破球逃脫（初始有 3 支針）

m_player_index 值	狀態	顯示圖片	備註
1	被水球困住	m_Balloon_player	顯示泡泡圈、無法操作
2	一般狀態	m_playPixmapX[] (上下左右)	根據方向顯示移動動畫
3	騎乘烏龜狀態	m_turtleUp/Right/ Down/Left	顯示烏龜上的角色動畫

- (4) trapStartTime.msecsTo(now) > 3000 若被水球困住超過 3 秒會失去 1 條命
- (5) void GameScene::checkPlayerCollideMonster():被 Monster 打到，會失去 1 條命
- (6) showGameOverAndExit(1):初始生命 3 條(player life = 3):歸零後進入 Game Over

2. BOSS 行為（class boss）

移動與互動

- (1) 主動朝玩家方向移動 updateBossAI();
- (2) 會根據玩家位置轉動面向（上／下／左／右）

(3) 可穿越所有障礙物

若碰到地圖上的道具，會直接使其消失



4. 玩家水球爆炸機制 (WaterBalloon class):

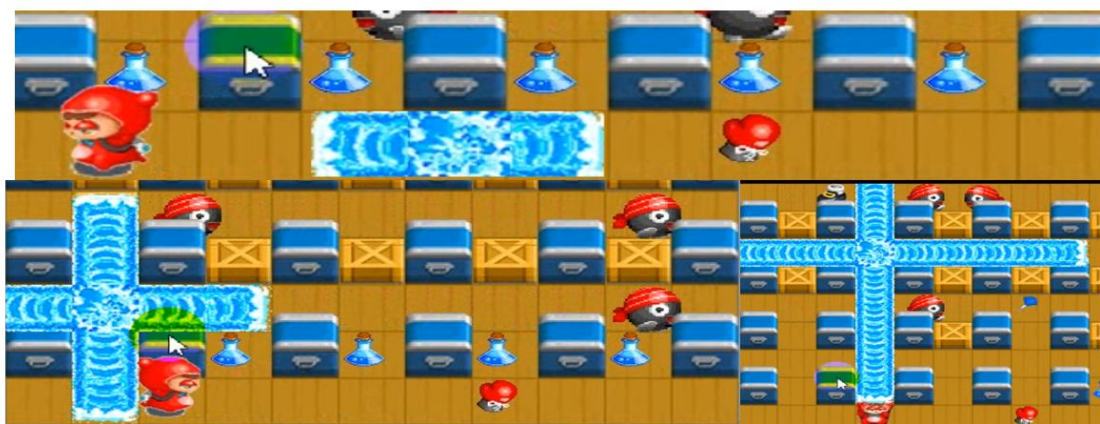
WaterBalloon 最多 5 顆

(1) `balloon.spawnTime.msecsTo(now) < 3000`: 每顆水球 3 秒後爆炸。

(2) `void GameScene::Draw_WaterColumn(WaterBalloon& balloon)`: 依據 `Water_Pow` 向上下左右延伸爆炸波。

`void GameScene::Draw_WaterColumn(WaterBalloon& balloon) =>` 堅固牆壁終止延伸 `if (type == 3) break`: 利用 `QTime` 追蹤放置時間，爆炸時判斷是否有磚塊或牆阻擋

(3) `for (WaterBalloon& other : m_STgame.m_StevenPeng.m_balloons)`: 支援連鎖爆炸，會檢查爆炸範圍內是否還有其他尚未爆炸水球，並一併引爆



透過道具 `Water_Pow` 延伸範圍造成上下左右各 1~5 格的爆炸傷害

如下若尚未爆炸的水球被其他爆炸波及，將會立即引爆可形成連鎖爆炸效果



3. 怪物與 Boss：

- (1) `initMonstersRound1()`:Round 1 初始化第 1 關怪物配置
- (2) `initMonstersRound2()`:Round 2 初始化第 2 關怪物配置
- (3) `initBoss()`:初始化 Boss 配置
- (4) 控制 BOSS 人工智慧（簡單追蹤玩家）`void`

`GameScene::updateBossAI()`:過程中若章魚碰到玩家水球，也會立刻爆炸並受傷

4. BOSS 的水球特性 (BossBalloon class)

BossBalloon 最多一顆

- (1) `void Player::fireBossBalloon(int direction)`: 直線前進
- (2) `if (m_STgame.m_StevenPeng.updateBossBalloons(playerRect))`: 接觸玩家時會立即爆炸
- (3) `if (m_STgame.m_StevenPeng.bossBalloons.at(0).spawnTime.msecsTo(now) < 3000)` 最多存在 3 秒後自動爆炸
- (4) `void Player::fireBossBalloon(int direction)` 遇到邊界會原路反彈
- (5) `void Player::touchbosspath(int direction)`: 遇到章魚本體會原路反彈



5. 勝利與失敗條件

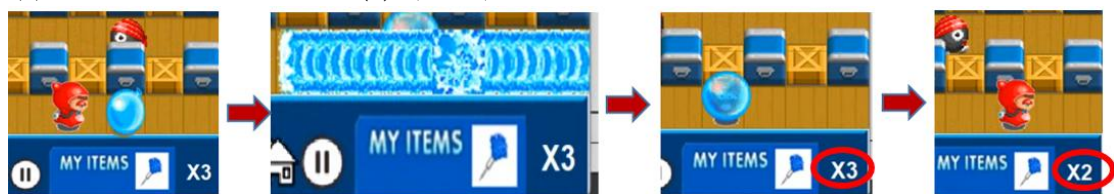
- (1) `void GameScene::checkBossHitByExplosion()`: 章魚必須被玩家的水球命中 6 次才會被擊敗
- (2) `void GameScene::updateBossAI()`: 若章魚碰到玩家水球，也會立刻爆炸並受傷
- (3) 擊敗章魚後玩家過關並進入勝利畫面

被玩家水球打到 6 次→死亡
player wins the round



6. 玩家失敗條件：

- (1) `trapStartTime.msecsTo(now) > 3000` 若被水球困住超過 3 秒或被章魚接觸，會失去 1 條命
- (2) `showGameOverAndExit(1)`: 命數歸零則 Game Over



玩家被水球包圍，按下 **Shift** 鍵針刺破水球避免損失生命

7. 使用者介面與互動設計

遊戲中使用 `drawUI()` 更新顯示資訊，包括：

- (1) 生命值（生命圖示與文字顯示）
- (2) 道具數（針、鞋子、水球數量）
- (3) 當前關卡提示（`roundindex` 控制）
- (4) 遊戲勝利或失敗彈窗提示（透過 `QMessageBox` 實作）
- (5) 此外還提供暫停與回主畫面按鈕，便於測試與切換模式。

遇到的挑戰與解決方式

在本次《Crazy Arcade (爆爆王)》專案中，因為在開發 project2 《Pokémon RPG》後，我對 Qt 架構、物件導向與場景控制有了基礎認識，這對《Crazy Arcade (爆爆王)》開發帶來幫助。RPG 專案中學習的 QGraphicsView 操作、物件互動與事件觸發，讓我能快速建構爆爆王中的地圖、角色、水球與道具管理系統。

但與 Pokémon RPG 相較，《Crazy Arcade (爆爆王)》為即時策略型遊戲，不同於 Pokémon 的回合制操作與對話互動。《爆爆王》強調玩家即時移動、水球放置與連鎖爆炸判斷，須同步處理動畫、爆炸影響與 AI 決策。技術上，Pokémon 多使用場景切換與資料框顯示，《爆爆王》則需統一場景中進行高頻率的物件更新與碰撞處理。整體而言，《爆爆王》在時間控制與即時互動邏輯的設計與挑戰度更高。

《Crazy Arcade (爆爆王)》在開發過程中面臨多項技術與計算挑戰，包括 AI 行為計算、道具系統與地圖變化效能、動畫和互動管理。本段落詳細說明在實作過程中所找解與改進的困難點，以重要技術難題為主，分項說明如下：

1. 水球連鎖爆炸與爆炸距離判斷背後合理化

最初的水球爆炸只有單一顆水球於尚未爆炸時的影響，並未考慮同時影響到其他未爆炸水球的情況，因此導致連鎖爆炸失效。為了解決這個問題，我在爆炸時添加對所有一幾格內未爆炸水球進行預先瀏覽，如果被爆炸過的格子已含有其他水球，則一並觸發其爆炸。

當初在設計水球連鎖爆炸時，未考慮爆炸路徑中的障礙物（如牆壁或箱子），導致即使有阻擋，仍會錯誤觸發後方水球爆炸。這在遊戲邏輯上不合理，也會破壞玩家策略判斷。後續修正時加入了「爆炸波必須無阻礙」的判斷，確保只有在同一方向且路徑暢通時才會連鎖引爆，提高遊戲公平性與真實感。

2. 地圖數據成長與合法性設計

首次針對 txt 格式地圖進行解析時，常常因為外部格式不符規範而無法解析，為了防止錯誤，我加入自動變化 string 表示線的 split 與 對各格子轉換 toInt()，如遇意外值則即時輸出錯訊，確保安全性。

3. 觸發互動與觀測更新形成相符化困難

最初 UI 或玩家狀態展示很常與實際對玩家行為不一致，包括殺敵后怪物未展示補助道具、被困時 UI 未顯示針數等問題。

為此，我把 UI 表示與 drawObj() 分離，在每次 QTimer 週期引動 drawUI() 根據現狀相關數據重新顯示。同時，每個展示層定義不同 zValue，確保展示優先級正確，並清除舊有的物件分層，避免重疊或不正確圖層顯示。

4. 機器人 AI 行為與 BFS 道路計算經歷多次重建

起初嘗試機器人行為只是簡單的直線移動到玩家起始為止，完全不考慮地圖構造與磚塊遮擋，可能遭遇無限加路或失效行為。之後將模型更換為 BFS 底層尋路，加入地圖類型判讀，但仍存在無法作出按境為出的連鎖投擲水球行為。

為了解決不同磚塊操作權限，我將 BFS 分成三階段判斷：(1)先依地圖情況主動重新排列操作路徑 (2)遇到可推磚，試著前移磚塊後重新 BFS (3)遇到無法投擲加路則以水球爆炸破壞，為最後攻略步驟

結合追蹤行為、特殊道路保留與預先最短路計算，將機器人行為設計成一個迴遷觀察與動態正覺調整系統，沒有發生無限遊躍。

總體而言，這些困難經過多輩調試、重建模型與加入我本身確認力後，成功轉化為經驗成熟、自動化度高的玩家互動遊戲系統。